HSD-001

Project Bootstrap and Directory Structure

Codex Implementation Task | Health Screening Offline Desktop

Task ID	HSD-001
Branch	feature/hsd-001-project-bootstrap
Commit message	chore: bootstrap offline desktop application
Primary stack	Electron + React + TypeScript + electron-vite + pnpm
Scope	Engineering foundation only; no clinical features or database

Execution rule: implement only this task, verify locally, push to GitHub, then request review before starting HSD-002.

1. Goal

Create a clean, reproducible Electron + React + TypeScript project that starts in development and builds successfully.

This task creates only the engineering foundation and repository structure. Do not implement clinical workflows, authentication, SQLite, IPC business operations, routing, Tailwind CSS, or the final UX.

2. Project Context

- This repository will implement a Windows-first, offline-first community hypertension screening desktop application.
- Electron main process will own trusted application logic.
- The preload process will expose a narrow typed API.
- The React renderer will contain presentation logic only.
- Shared TypeScript contracts will sit outside process-specific code.
- SQLite and Windows packaging will be added in later tasks.

3. Package Manager and Scaffolding

Use pnpm and the official electron-vite quick-start React TypeScript template.

```
pnpm create @quick-start/electron health-screening-desktop --template react-ts
```

If Codex is already operating inside an initialized repository, scaffold into a temporary directory and move the generated files into the repository root. Do not create a nested application directory.

4. Repository Name and Branch

Repository	health-screening-desktop
Branch	feature/hsd-001-project-bootstrap

5. Target Directory Structure

```
health-screening-desktop/
├── .github/
│   └── workflows/
├── build/
│   └── installer-assets/
├── docs/
│   ├── adr/
│   ├── clinical-workflow/
│   ├── database/
│   ├── ipc/
│   ├── security/
│   └── testing/
├── resources/
│   ├── protocols/
│   ├── print-templates/
│   └── seed-data/
├── src/
│   ├── main/
│   │   ├── app/
│   │   ├── auth/
│   │   ├── config/
│   │   ├── database/
│   │   │   ├── migrations/
│   │   │   └── repositories/
```

```
│   │   ├── ipc/
│   │   │   └── handlers/
│   │   ├── services/
│   │   │   ├── patients/
│   │   │   ├── sessions/
│   │   │   ├── screenings/
│   │   │   ├── referrals/
│   │   │   ├── followups/
│   │   │   ├── reports/
│   │   │   ├── sync/
│   │   │   └── backup/
│   │   ├── protocol/
│   │   ├── printing/
│   │   ├── logging/
│   │   └── security/
│   ├── preload/
│   ├── renderer/
│   │   ├── index.html
│   │   └── src/
│   │       ├── app/
│   │       ├── components/
│   │       ├── features/
│   │       │   ├── auth/
│   │       │   ├── dashboard/
│   │       │   ├── patients/
│   │       │   ├── sessions/
│   │       │   ├── screenings/
│   │       │   ├── referrals/
│   │       │   ├── followups/
│   │       │   ├── reports/
│   │       │   ├── sync/
│   │       │   └── settings/
│   │       ├── routes/
│   │       ├── stores/
│   │       └── styles/
│   └── shared/
│       ├── contracts/
│       ├── schemas/
│       ├── enums/
│       ├── errors/
│       └── types/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── fixtures/
│   └── helpers/
├── electron.vite.config.ts
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
└── README.md
```

6. Implementation Requirements

1. Scaffold a working Electron + React + TypeScript application.
2. Preserve the electron-vite main, preload, and renderer entry points.
3. Move or adjust generated files to match the target structure.
4. Update imports and electron-vite configuration after moving files.
5. Ensure no source file uses JavaScript when TypeScript is appropriate.

6. Keep TypeScript configuration compatible with the main, preload, and renderer processes.
7. Add the packageManager field to package.json using the installed pnpm version.
8. Add these scripts if the template does not already provide equivalents: dev, build, start or preview, and typecheck.
9. Create docs/repository-structure.md explaining main-process, preload, renderer, shared-contract, and test-directory responsibilities.
10. Add README files or .gitkeep files so intentionally empty architecture directories are committed.
11. Replace the default demo interface with a simple placeholder page containing the application name, status “Engineering foundation,” and a statement that no clinical features are implemented yet.
12. Remove unused template images, demo counters, links, and sample components.
13. Do not add a database.
14. Do not expose raw ipcRenderer to the renderer.
15. Do not implement generic execute, send, filesystem, or shell APIs.
16. Do not add remote URLs or remotely loaded content.
17. Do not add production credentials or environment secrets.
18. Keep dependencies limited to those needed by the generated baseline.

7. Files to Review Carefully

- package.json
- electron.vite.config.ts
- TypeScript configuration files
- src/main entry point
- src/preload entry point
- src/renderer/index.html
- src/renderer/src/main.tsx
- src/renderer/src/App.tsx
- README.md
- docs/repository-structure.md

8. Verification and Tests

A full automated test framework is not required in this task. At minimum, verify the following:

1. TypeScript typechecking succeeds.
2. The development application launches.
3. The production build completes.
4. The renderer shows the new placeholder page.
5. The application opens only one main window.
6. No Node.js API is directly available in renderer code.

9. Commands to Run

```
pnpm install
pnpm typecheck
pnpm build
pnpm dev
```

If the scaffold uses a different valid typecheck command, document it in README.md and package.json.

10. Acceptance Criteria

- A clean checkout can install dependencies with pnpm.
- pnpm typecheck completes without errors.
- pnpm build completes without errors.
- pnpm dev launches the Electron application.
- The application shows the Health Screening Offline Desktop placeholder.
- The target directory structure exists and is committed.
- Generated demo content has been removed.
- Main, preload, renderer, and shared concerns are visibly separated.
- No clinical feature, authentication feature, database, or business IPC operation has been implemented.
- README.md contains setup and run instructions.
- docs/repository-structure.md documents the architecture boundaries.

11. Out of Scope

- Tailwind CSS
- Final UX styling
- Top navigation
- React Router
- Zustand
- Zod
- React Hook Form
- SQLite or better-sqlite3
- Local authentication
- Clinical workflow
- Patient tabs
- CI workflow
- Packaging
- Synchronization
- Test-framework configuration beyond what the scaffold already includes

12. Completion Evidence Required from Codex

1. Final directory tree.
2. List of files created, moved, or modified.
3. Package scripts.
4. Command results for typecheck and build.
5. Screenshot or description of the launched placeholder window.
6. Any scaffold deviation and the reason.
7. Confirmation that no functionality outside this task was added.

13. Commit Message

```
chore: bootstrap offline desktop application
```

14. Review Gate

Do not begin HSD-002 until HSD-001 has been pushed to GitHub and reviewed.

Provide the repository URL, branch name, and commit hash for review. Any requested corrections should be completed on the same branch before the task is approved.